



了解JavaFX Bean Property技术特性

北京理工大学计算机学院
金旭亮

学习导引



JavaFX数据绑定技术的根基，是JavaFX Bean Property。



JavaFX Bean Property在JavaBean规范的基础上扩充而来，专为实现JavaFX数据绑定技术特性而设计。



在JavaFX Bean Property之前，应该先了解JavaBean规范，可以参看本课程介绍JavaBean的那个PPT，之后再学习本讲内容。

在JavaFX中

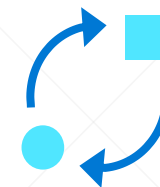


所有JavaFX控件，都定义有许多“可观察（Observable）”的属性，这些属性，都是JavaFX Bean Property。



这些属性支持“数据绑定（Data Bind）”。

JavaFX Property有哪些与众不同之处?



JavaFX的各个内置控件，其属性很多都是JavaFX Bean Property，这种类型的属性，在内部保存了控件相应属性值的当前状态，外界可以监听这些值，当这些值有变化时，注册的外部监听器对象会得到通知。



如果JavaFX控件某属性绑定到数据对象的某属性上，而这个数据对象的属性也是JavaFX Bean Property，那么在数据对象属性有变化时，JavaFX控件就会得到通知，从而可以实现自动刷新显示。



反过来，如果JavaFX控件属性发生了改变（比如用户在文本框中修改了值），那么这个新值也可以更新它所绑定的数据对象的相应属性。

JavaFX数据绑定技术的根基是JavaFX Bean Property

使用JavaFX Bean Property，我们可以很容易地实现UI控件与底层数据源对象之间的同步（这就是数据绑定技术应用的主要场景），能大大地减少需要编写的代码量。

数据绑定技术，现在几乎已经成为拥有可视化界面的应用程序开发框架的标配。除了诸如JavaFX、WPF等桌面应用开发框架，Android应用开发、Web前端页面开发等技术领域，都有相应的数据绑定组件或框架可用。



学会一个，即可“举一反三”。

现成可用的JavaFX Bean Property



在`javafx.beans.property`包中，定义有一些类和接口，可以基于它们定义自己的JavaFX Bean Property:

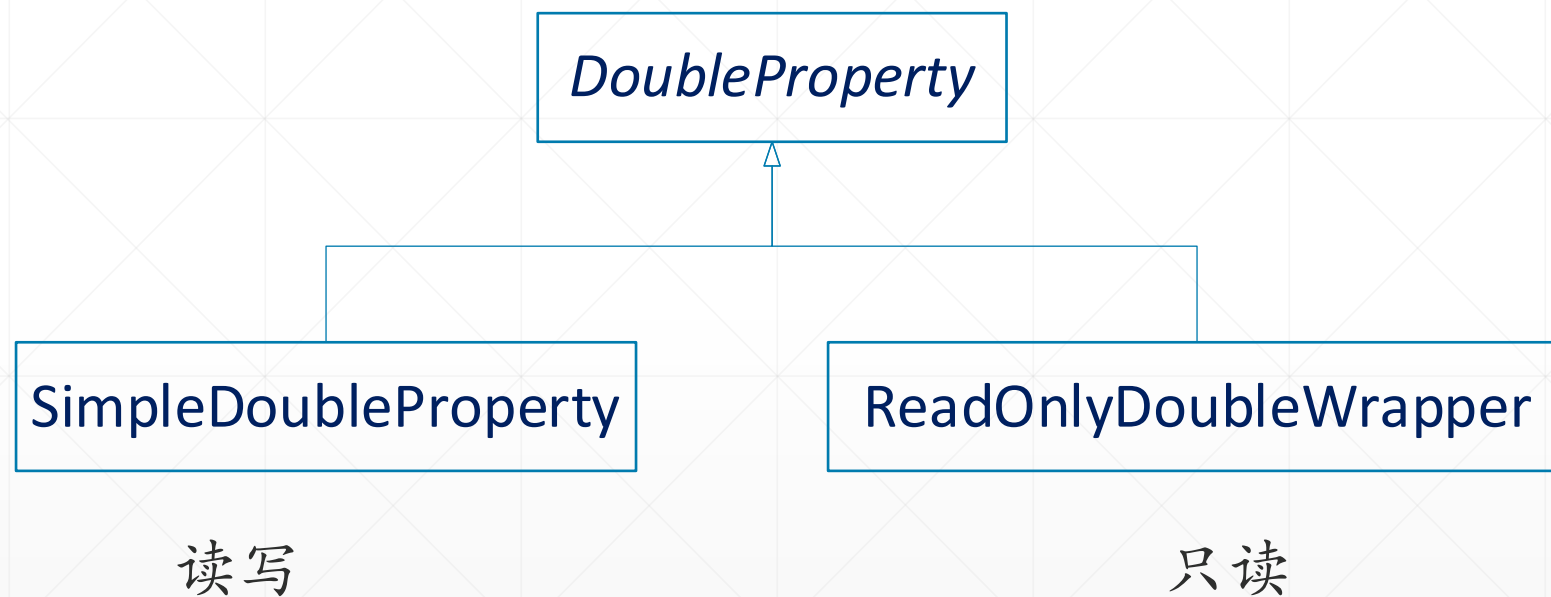
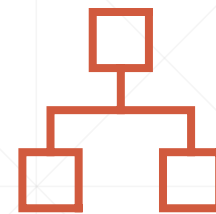
`SimpleStringProperty`
`SimpleDoubleProperty`
`SimpleIntegerProperty`
`SimpleBooleanProperty`

可读可写

`ReadOnlyBooleanWrapper`
`ReadOnlyIntegerWrapper`
`ReadOnlyDoubleWrapper`
`ReadOnlyStringWrapper`

只读

JavaFX Bean Property类型之间的继承关系



系统内置JavaFX Bean Property示例代码

ExploreJavaFXProperties.java

```
// 使用系统内置的JavaFX Bean Property
1 usage
private static void useJDKJavaFXProperties() {
    // 可读可写的属性
    StringProperty strProp = new SimpleStringProperty();
    strProp.set("Hello");
    System.out.println(strProp.get());
    // 只读的属性
    ReadOnlyStringWrapper wrapper = new ReadOnlyStringWrapper( initialValue: "World");
    ReadOnlyStringProperty readOnlyStringProperty = wrapper.getReadOnlyProperty();
    System.out.println(readOnlyStringProperty.getValue());
}
```



在JavaFX属性与绑定框架（JavaFX properties and bindings framework）中，各种类和接口总是同时有Boolean、Integer、Long、Float、Double、String和Object几个“版本”。

get/set方法总是有两套：get()和set()，getValue()和setValue()。

JavaFX Bean Property的构成



包容有一个私有变量，用于存储数据。



定义有公有的getter和setter方法，用于存取这个私有数据。



定义有一个默认的构造方法。



可以触发事件和数据值更改通知。

MyJavaBean

```
-x: IntegerProperty  
+MyJavaBean()  
+getX(): int  
+setX(x: int): void  
+xProperty(): IntegerProperty
```

JavaFX Properties 事件响应



可以给JavaFX Bean Property挂接两种类型的监听器对象：InvalidationListener或ChangeListener。



给JavaFX Bean Property赋值时，如果新值与旧值不一样，这些监听器对象都会得到通知。

```
@FunctionalInterface
public interface InvalidationListener {
    public void invalidated(Observable observable);
}
```

```
@FunctionalInterface
public interface ChangeListener<T> {
    void changed(ObservableValue<? extends T> observable, T oldValue, T newValue);
}
```

被赋值的属性

旧值

新值

可观察的类型—Observable



InvalidationListener接口所定义的invalidated()方法的参数，其声明的类型为**Observable**，这是一个抽象类，并不包容太多的信息。它的真实类型，是由被监听的JavaFX Bean Property类型所决定的，所以，如果想提取出属性的相关信息，必须进行类型强制转换，才能提取出它所包容的数据：

```
rotate.statusProperty().addListener(observable -> {  
    text2.setText("Animation status: " +  
        ((ObservableObjectValue<Animation.Status>)observable)  
        .getValue());  
});
```

理解invalidate



默认情况下，JavaFX Bean Property采用“**延迟计算**”的策略。



也就是说，JavaFX Bean Property所依赖的值发生变化时，它不会立即更新以反映这种变化，而是会设置内部的一个标记，表明当前数据“无效”，需要重新计算。



当外界调用JavaFX Bean Property的get()或getValue()方法时，其值才会更新，这时，此属性值才是“**有效 (valid)**”的。

添加JavaFX Bean Property监听器



```
// 测试JavaFX Bean Property的属性值更改通知(InvalidationListener)
1 usage
private static void addAndRemoveInvalidationListener() {
    // 创建一个JavaFX Property
    var intProperty = new SimpleIntegerProperty( initialValue: 1024);
    // 创建一个 InvalidationListener监听器对象
    final InvalidationListener invalidationListener =
        observable ->
            System.out.println("值有变化, observable对象处于invalidated状态");
    // 挂接监听器对象到JavaFX Property
    intProperty.addListener(invalidationListener);
    System.out.println("已经添加invalidation监听器。");
}
```

测试值更改的时机



//首次赋值测试

```
System.out.println("调用intProperty.set(2048), 将触发“值改变”通知。");  
intProperty.set(2048);
```

//不读取值而多次赋值

```
System.out.println("调用intProperty.setValue(3072), 不会触发值改变通知。");  
intProperty.setValue(3072);  
System.out.println("调用intProperty.setValue(4096), 不会触发值改变通知。");  
intProperty.setValue(4096);  
System.out.println("提取当前值: "+intProperty.get());  
System.out.println("调用intProperty.setValue(5120), 将触发“值改变”通知。");  
intProperty.setValue(5120);  
System.out.println("提取当前值: "+intProperty.get());
```

//现在移除监听器对象

```
System.out.println("已移除invalidation监听器。");  
intProperty.removeListener(invalidationListener);
```

// 再次赋值测试

```
System.out.println("调用intProperty.set(4096), 已无监听器可供响应。");  
intProperty.set(4096);
```

```
}
```

```
Run  [C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" ...  
已经添加invalidation监听器。  
调用intProperty.set(2048), 将触发“值改变”通知。  
值有变化, observable对象处于invalidated状态  
调用intProperty.setValue(3072), 不会触发值改变通知。  
调用intProperty.setValue(4096), 不会触发值改变通知。  
提取当前值: 4096  
调用intProperty.setValue(5120), 将触发“值改变”通知。  
值有变化, observable对象处于invalidated状态  
提取当前值: 5120  
已移除invalidation监听器。  
调用intProperty.set(4096), 已无监听器可供响应。  
  
Process finished with exit code 0
```

试验小结



当JavaFX Bean Property居于“invalidated”状态时，后继对它调用set（或setValue）方法进行多次赋值时，不会再触发Invalid事件。只有等到调用get方法时，让属性居于validated状态之后，此事件才会再次触发。

测试ChangeListener-1



```
private static void addAndRemoveChangeListener() {  
    // 创建一个JavaFX Property  
    var intProperty = new SimpleIntegerProperty( initialValue: 1024);  
  
    // 创建一个 InvalidationListener监听器对象  
    final ChangeListener changeListener =  
        (ObservableValue observableValue,  
         Object oldValue,  
         Object newValue) ->  
            System.out.println("observableValue已更改: 老值 = "  
                               + oldValue + ",新值 = " + newValue);  
  
    // 挂接监听器对象到JavaFX Property  
    intProperty.addListener(changeListener);  
    System.out.println("值更改监听器已添加。");  
}
```

ChangeListener可以获取
新值和旧值

测试ChangeListener-2



测试代码：

```
// 赋值测试
System.out.println("调用 intProperty.set(5120), 监听器将响应");
intProperty.set(5120);

// 移除监听器对象
intProperty.removeListener(changeListener);
System.out.println("值更改监听器已被移除。");

// 再次赋值测试
System.out.println("此时调用intProperty.set(6144), 无人监听。");
intProperty.set(6144);
```

运行结果：

```
Run
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
值更改监听器已添加。
调用 intProperty.set(5120), 监听器将响应
observableValue已更改: 老值 = 1024,新值 = 5120
值更改监听器已被移除。
此时调用intProperty.set(6144), 无人监听。

Process finished with exit code 0
```



仅仅只有赋的值不同时，才会触发相应的事件。这就是说，如果你给一个 JavaFX Bean Property 赋相同的值多次，只有第一次会触发事件，请自行测试一下这个技术特性。